

图及其遍历

1. 基本概念

(1) 邻接矩阵: $G = (V, E), |V| = n, |E| = m$

G 的邻接矩阵就是一个 $n \times n$ 的矩阵, 记为 $A = (a_{ij})$

$$\text{其中 } a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}$$

如果 G 是无向图, 则其邻接矩阵是对称的。

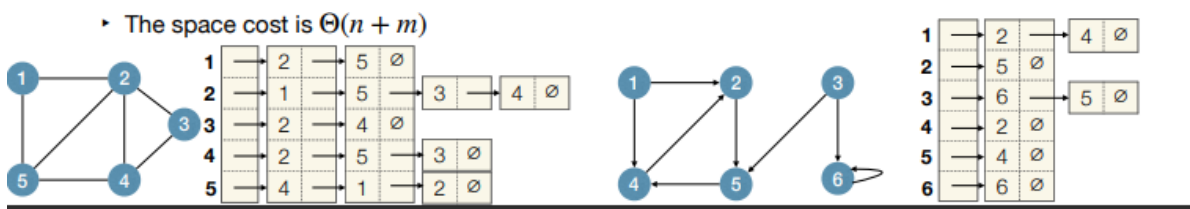
内存消耗为 $\Theta(n^2)$ 。

(2) 邻接链表: $G = (V, E), |V| = n, |E| = m$

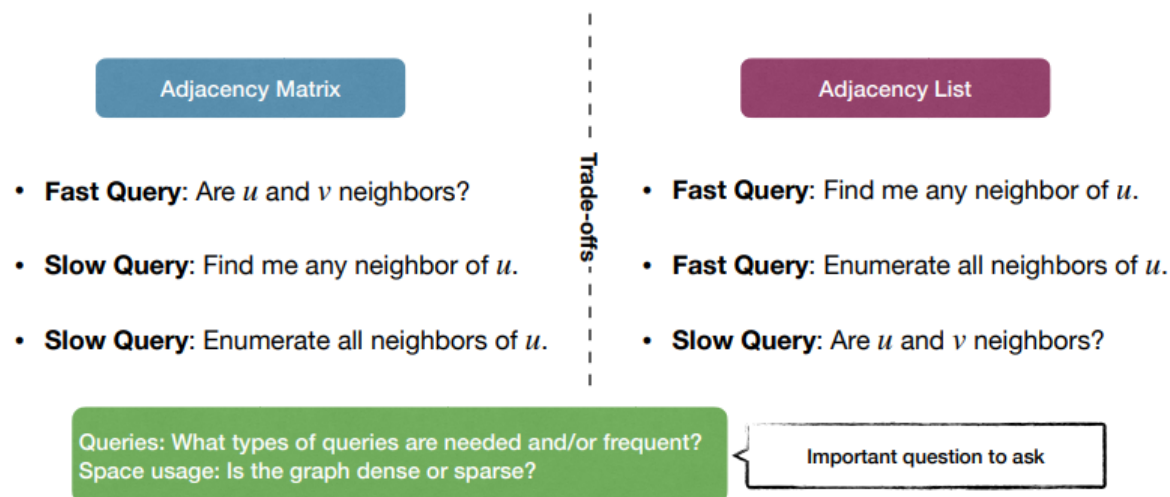
G 的邻接链表是 n 个链表和集合, 对于每一个顶点 $u \in V$, 在 u 的链表中, 当且仅当边 $(u, v) \in E$ 时顶点 v 存在该链表中。

如果 G 是无向图, 则每条边都出现两次。

空间复杂度为 $\Theta(n + m)$ 。



(3) 邻接矩阵和邻接链表的比较:

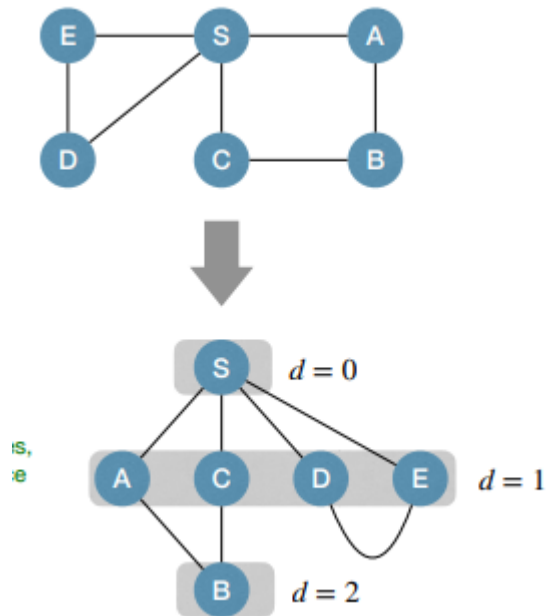


2. 图的遍历

2.1 Breadth-First Search(BFS)

基本思想: 从起始节点 s 开始, 按层访问其他的节点 (从 s 可到达)。

使用一个 *FIFO* 队列



```

1  BFSkeleton(G, s):
2  for each u in V
3      u.dist := INF, u.discovered := False
4  s.dist := 0, s.discovered := True
5  Q.enqueue(s)
6  while !Q.empty()
7      u := Q.dequeue()
8      for each edge (u, v) in E
9          if !v.discovered
10             v.dist := u.dist + 1
11             v.discovered := True
12             Q.enqueue(v)

```

节点有三种状态：

- 未被发现的（白色节点）：仍未在队列中
- 发现了但是没有访问的（灰色节点）：在队列中但是没有处理
- 访问了的（黑色节点）：弹出队列并且处理过了

```

1  BFS(G, s):
2  for each u in V
3      u.c := WHITE, u.d := INF, u.p := NIL
4  s.c := GRAY, s.d := 0, s.p := NIL
5  Q.enqueue(s)
6  while !Q.empty()
7      u := Q.dequeue()
8      u.c := BLACK
9      for each edge (u, v) in E
10         if v.c == WHITE
11             v.c := GRAY
12             v.d := u.d + 1
13             v.p := u
14             Q.enqueue(v)

```

BFS 的分析：（对于 G 是连通的）

- 时间复杂度：
 - “While”循环需要 $\Theta(n)$, Q 中的每个节点最多经过一次
 - “For”循环需要 $\Theta(m)$, 每条边最多访问了一次或两次
- BFS 的时间复杂度为 $\Theta(n + m)$

如果 G 不连通, 那么就在每一个连通分量中进行 BFS

2.2 Depth-First Search(DFS)

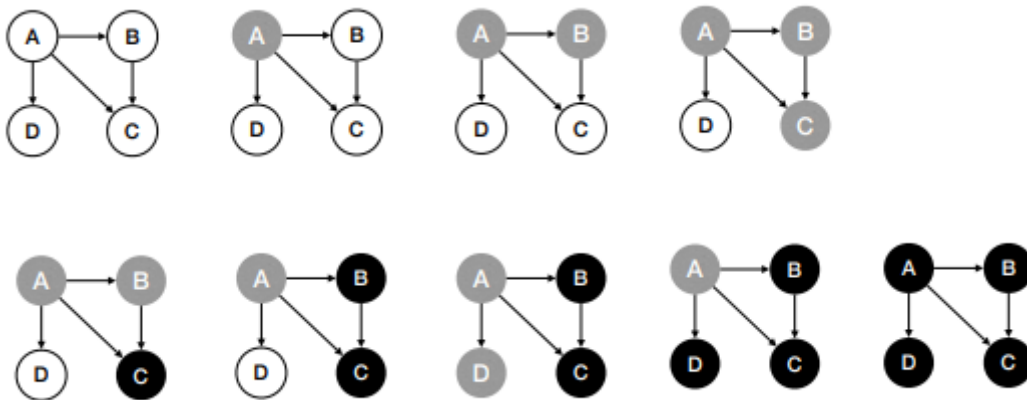
类比走迷宫, 碰壁或者遇到已经走过的路就回头, 直到找到出口

使用栈

```

1 DFSkeleton(G, s):
2   s.visited := True
3   for each edge (s, v) in E
4     if !v.visited
5       DFSkeleton(G, v)

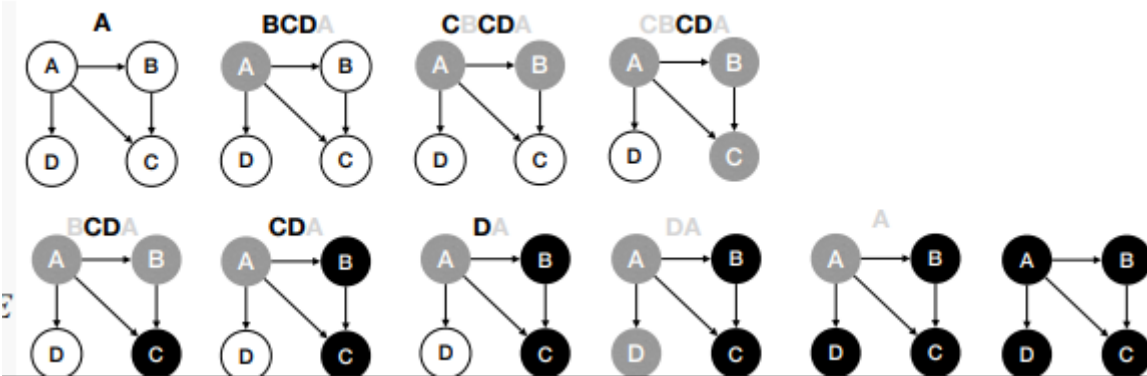
```



```

1 DFSIterSkeleton(G, s):
2   Stack Q
3   Q.push(s)
4   while !Q.empty()
5     u := Q.pop()
6     if !u.visited
7       for each edge (u, v) in E
8         Q.push(v)

```



如果 G 是不连通的呢?

从多个源节点进行 *DFS*

只需在上面的代码前面补充以下代码：

```
1 DFSAll(G):
2   for each node u in V
3       v.visited := False
4   for each node u in V
5       if !u.visited
6           DFSSkeleton(G, u)
7           //DFSIterSkeleton(G,u)
```

节点有三种状态：

- 未被发现的（白色节点）：在调用 $DFSSkeleton(G, u)$ 之前
- 发现的（灰色节点）：在执行 $DFSSkeleton(G, u)$ 中
- 结束的（黑色节点）：在 $DFSSkeleton(G, u)$ 返回时

$DFS(G, u)$ 以 u 为根建立了一个树， DFS 在整个图上建立了一个森林

```
1 DFSAll(G):
2   for each node u in V
3       u.color := WHITE
4       u.parent := NIL
5   for each node u in V
6       if u.color = WHITE
7           DFS(G, u)
8
9 DFS(G, s):
10  s.color := GRAY
11  for each edge (s, v) in E
12      if v.color = WHITE
13          v.parent := s
14          DFS(G, v)
15  s.color := BLACK
```

对 *DFS* 的分析：

1. *DFS* 在处理每个节点时至少提供了两次机会
 - $Pre - visit : WHITE \rightarrow GRAY$
 - $Post - visit : GRAY \rightarrow BLACK$
2. 示例应用程序：跟踪节点的活动时间间隔。当某些节点的颜色发生变化时，时钟就会发生变化。
 - 发现时间戳：当节点变为灰色时
 - 结束时间戳：当节点变为黑色时

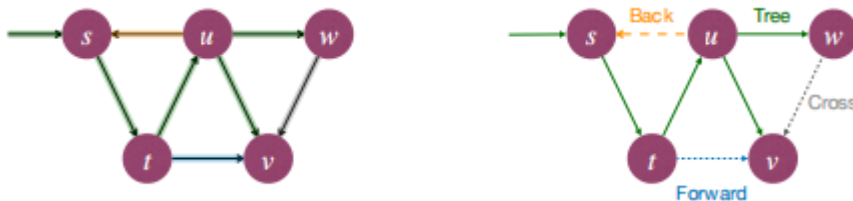
加上时间戳后的代码修改如下：

<pre> DFSAll(G): PreProcess(G) for each node u in V $u.color := WHITE$ $u.parent := NIL$ for each node u in V if $!u.visited$ DFS(G, u) </pre>	<pre> DFS(G, s): PreVisit(s) $s.color := GRAY$ for each edge (s, v) in E if $v.color = WHITE$ $v.parent := s$ DFS(G, v) $s.color := BLACK$ PostVisit(s) </pre>	<pre> PreProcess(G): time := 0 PreVisit(s): time := time + 1 $s.d := time$ PostProcess(G): time := time + 1 $s.f := time$ </pre>
--	--	---

Note: here it indicates the discovery time

3. 边的分类：

- 树边：在 DFS 森林中的边
- 后向边：在一个 DFS 树中边 (u, v) 连接了 u 到其祖先 v
- 前向边：不是树的边 (u, v) ,在一个 DFS 树中连接了 u 和它的后代 v
- 交叉边：其他的边



2.3 深度优先搜索的特性

1. Parenthesis Theorem(括号定理)：两个节点的活动间隔要么是：(a)完全不相交；或者(b)一个完全包含在另一个节点中。

对于任意两个节点 u 和 v ，满足以下一个情况：

- $[u.d, u.f]$ 和 $[v.d, v.f]$ 是不相交的，在 DFS 森林中，没有祖先-后代关系
- $[u.d, u.f] \subseteq [v.d, v.f]$, 且 u 是 v 的一个后代
- $[v.d, v.f] \subseteq [u.d, u.f]$, 且 u 是 v 的一个祖先

证明：

- 若 $u.d < v.d$,
 - 若 $v.d < u.f$, 即 v 在 u 为灰色的时间段里被发现，意味着 v 是 u 的后代，有 $v.f < u.f$, 即 $[u.d, u.f] \subseteq [v.d, v.f]$
 - 若 $v.d > u.f$, 即 v 在 u 为黑色后被发现，有 $u.d < u.f < v.d < v.f$, 即 $[u.d, u.f] \cap [v.d, v.f] = \emptyset$
- 若 $v.d < u.d$,
 - 若 $u.d < v.f$, 即 u 在 v 灰色的时间段里被发现，意味着 u 是 v 的后代，有 $u.f < v.f$, 即 $[v.d, v.f] \subseteq [u.d, u.f]$
 - 若 $u.d > v.f$, 即 u 在 v 为黑色后被发现，有 $v.d < v.f < u.d < u.f$, 即 $[v.d, v.f] \cap [u.d, u.f] = \emptyset$

2. White-path Theorem(白色定理)：在有向图或无向图 $G = (V, E)$ 的深度优先森林中，顶点 v 是顶点 u 的后代当且仅当在搜索发现 u 的时刻 $u.d$ ，存在一条从 u 到 v 的全部由白色顶点构成的路径。

3. 在对无向图 $G = (V, E)$ 的深度优先探索中, G 中的每一条边要么是树边, 要么是后向边。